Gem Language Tutorial Series

David B Hon

2026-04-16

Gem Language Tutorial Series

Version: 0.1.0 | **Date:** 2026-04-16 | **Built with:** Gemini CLI & Kiro CLI

Welcome to the official tutorial repository for the **Gem Language** — a modern, expressive STEM language built on C++26 for scientific computing, AI, finance, and cross-platform application development.

Key Features

- **Explicit Typing with Initial Values**: All variable declarations require an initial value (`int x = 0`).
- **Global vs. Local Scope**: Variables starting with `_` (e.g., `_config`) are global; all others are local.
- **Universal Inheritance**: Every object inherits from `sys`, giving immediate access to `sys.print`, etc.
- **First-Class AI Support**: Built-in `ai` object supports Gemini, Mistral, and Ollama.
- **Polyglot Interop**: `use` keyword bridges Python, Julia, R, Fortran, C++, Go, Ruby, Rust, Node.js on-the-fly.
- **Mobile & Cross-Platform**: `mobl` object + browser PWA (Web Speech + Geolocation) works on Android, iPhone, macOS, Linux, and Windows 11.
- **Domain-Specific Built-ins**: `fin`, `bsm`, `bev`, `geo`, `data`, `chart`, `mobl`, `trek`, `astro`, `nlp`, `www`, `cdn`, `thread` built into the runtime.
- **Regular Expressions**: `rex` builtin provides full ECMAScript regex — match, find, findall, groups, sub, gsub, split, count.
- **Symbolic Math**: `math` builtin supports symbolic differentiation, integration, simplification, and LaTeX output via SymPy/Sage.
- **Astrophysics**: `astro` builtin covers stellar physics, orbital mechanics, cosmology, solar physics, and exoplanets.

Tutorial Roadmap

1. The Foundations

- **01_basics.g**: Variables, types, and basic I/O using the `sys` object.
- **02_math_latex.g**: Arithmetic operations and LaTeX-style mathematical expressions.
- **03_text_pdf.g**: Text manipulation and PDF document generation.
- **04_images.g**: Image processing with ImageMagick.
- **05_geo_gis.g**: GIS, geolocation, GeoJSON, and interactive maps.

2. Application Development

- **06_web_apps.g**: Building web servers with `sys.app()` and route handlers.
- **07_ai_nlp.g**: NLP and AI prompting with Gemini, Mistral, and Ollama.
- **08_functions.gm**: Defining functions, objects, and inheritance.
- **09_module_demo.g**: Structuring code with modules.

Gem v0.1.0 2026-04-16

- **11_modules_paths.g**: Managing module resolution and import paths.

3. Advanced Integration & AI

- **10_interop_cpp.g**: High-performance C++ interoperability via Cling JIT.
- **12_lib_management.g**: Dependency management and library linking.
- **13_dependency_details.g**: Deep dive into Gem's package ecosystem.
- **14_ai_providers.g**: Using Gemini, Mistral, and Ollama side-by-side.
- **15_mistral_native.g**: Native Mistral C++ bridge via `prompt_native`.

4. Specialized Tooling

- **16_algo_text_extras.g**: Advanced string algorithms and text analysis.
- **17_repl_features.g**: Mastering the interactive Gem REPL.
- **18_tcp_sockets.g**: Low-level networking fundamentals.
- **19_tcp_server.g** & **20_tcp_client.g**: Client-server architectures.
- **21_polyglot.g**: Bridging Python, Julia, and Gem.

5. Science, Finance & DevOps

- **22_builtins_itr_tcp.g**: Iterators and network protocols.
- **23_data_science.g**: DataFrames, statistics, and visualization.
- **24_devops.g**: Automation scripts and infrastructure management.
- **25_ollama_mistral.g**: Running local LLMs via Ollama.
- **26_devops_containers_vms.g**: Docker and Vagrant environments.

6. High-Performance & Symbolic Math

- **27_rust_node.g**: Polyglot support for Rust and Node.js.
- **28_interactive_charts.g**: Dynamic 2D/3D/multi-axes Plotly visualization.
- **29_quantlib_finance.g**: Quantitative finance with QuantLib.
- **30_bsm_pde_inheritance.g**: Black-Scholes-Merton PDE solvers.
- **31_history_and_compilation.g**: Gem's compilation model and history.
- **32_symbolic_math.g**: Symbolic differentiation and integration.
- **33_langport_porting.g**: Porting legacy codebases to Gem.
- **34_geo_tectonics.g**: Advanced GIS, tectonic modeling, and geo-visualization.
- **35_text_advanced.g**: Large-scale text processing and indexing.

7. Mobile & Cross-Platform

- **36_mobl_travel_log.g**: `mobl` object — NLP dictation + GPS + GeoJSON travel log PWA.

8. System Programming & Hardware

- **37_device_drivers.g**: `drvr` builtin — Linux, Windows 11, macOS, and Android device driver development.

9. Text & Pattern Matching

- **38_rex.g**: `rex` builtin — full ECMAScript regular expressions: match, find, findall, groups, sub, gsub, split, count.

10. Travel Logs & OSM Mapping

- **39_trek.g**: `trek` builtin — create, display, and edit travel logs on OpenStreetMap with GeoJSON/GPX support.

Gem Language v0.1.0 2026-04-16

11. Search Engine Optimization

- **40_seo.g**: `seo` builtin — crawl URLs, extract SEO signals, write JSON index, analyze report.

Help & Builtins

```
--- Gem Language Help ---
Gem is a high-performance STEM language implemented in C++26.

Usage Tips:
  - All variables must be initialized: 'int x = 0'
  - Global variables start with underscore: '_config = 1'
  - Use 'use "file.py"' to translate and run foreign code.
  - Use 'langport("*.py", "out.gm")' to port and save code.
  - Use 'lib' to see all loaded modules.
  - Use 'his' to view today's session history.

CLI Options:
  gem <file.g>              - Run a script.
  gem -c <main.g> [mod...]  - Compile multiple files into a binary.
  gem -o <name>             - Specify output name for compiler.
  gem -h                    - Print history and exit.
  gem -t <file> [-o output] - AI-assisted translation to Gem.

Available Builtin Modules:
  sys, math, ai, text, rex, algo, bev, file, zip, nlp, img, www, cdn, geo,
  cpp, tcp, itr, thread, data, k3s, vm, go, ruby, node, rust,
  fin, bsm, chart, astro, mobl, trek, seo, drvr

Keywords for Documentation:
  fun, obj, use, alias, his, lib, end, if, while,
  int, double, string, bool, exit

Mobile & Cross-Platform:
  mobl phone("name")  phone.dictate(text)  phone.make_feature(lat,lon,text)
  ./gem travel_log.g  ->  http://localhost:8080  (Android/iPhone/macOS/Linux/Win11)

Detailed help: help "topic"  or  help(topic)
```

Builtin Module Reference

sys — System Root Module

Every object inherits from `sys`. Methods:

- `sys.print(...)` — print to stdout
- `sys.args()` — command-line arguments as a vector
- `sys.async(func, [params], [timeout])` — run function in background thread
- `sys.sethandler(signals, func)` — set custom signal handler
- `sys.host()` — OS information string
- `sys.exec(cmd)` — execute a shell command
- `sys.doc([path])` — read file comments or current script comments
- `sys.help([topic])` — interactive help

Gem v0.1.0 · 2026-04-16

- `sys.exit()` — exit the REPL or script
- `sys.langport(pattern, [output])` — port foreign code (e.g. `*.py`) to Gem
- `sys.redirect(url, [port])` — HTTP redirect or background redirect server
- `sys.app(port, [routes])` — background web app server; routes map paths to strings, files, or handler functions

math — Mathematics & Symbolic Math

Standard functions: `sin(x)`, `cos(x)`, `sqrt(x)`, constant `math.pi`.

Symbolic math (SymPy/Sage backend):

- `math.useSymPy()`, `math.useSage()` — switch symbolic provider
- `math.simplify(expr)` — symbolic simplification
- `math.diff(expr, [var])` — symbolic differentiation
- `math.integrate(expr, [var])` — symbolic integration
- `math.solve(expr, [var])` — solve algebraic equation
- `math.sym_latex(expr)` — LaTeX for a symbolic expression
- `math.to_latex(val)` — convert value or matrix to LaTeX string
- `math.write_latex(path, content, [standalone])` — write LaTeX to file
- `math.read_latex(path)` — read content from LaTeX document environment
- `math.parse_latex(text)` — extract math expressions from text (`$...$`)
- `math.compile_latex(path)` — run pdflatex on a file

text — String & Document Processing

- `text.read(path)`, `text.sub(s, start, [len])`, `text.replace(s, old, new)`
- `text.write_pdf(path, text)`, `text.write_pdf_a(path, text)`, `text.read_pdf(path)`
- `text.read_markdown(path)`, `text.write_markdown(path, text)`
- `text.read_yaml(path)`, `text.write_yaml(path, obj)`
- `text.read_html(path)`, `text.write_html(path, text)`
- `text.read_xml(path)`, `text.write_xml(path, text)`
- `text.read_fits_header(path)` — read FITS file header as object
- `text.read_hdf_header(path)` — read HDF5 file header as object

geo — GIS & Geolocation

- `geo.lookup()` — IP-based geolocation (`.lat`, `.lon`, `.city`, `.country`)
- `geo.distance(lat1, lon1, lat2, lon2)` — Haversine distance in km
- `geo.write_geojson(path, features)`, `geo.history(plate)`
- `geo.plot2d(data, [layout])` — 2D Plotly map (open-street-map / scattermapbox)
- `geo.plot3d(data, [layout])` — 3D globe visualization (orthographic scattergeo)

astro — Astrophysics & Planetary Science

Constants: `G`, `c`, `AU`, `pc`, `ly`, `Msun`, `Rsun`, `Lsun`, `Tsun`, `Mearth`, `Rearth`, `H0`, `sigma_sb`

Stellar: `luminosity`, `stefan_boltzmann`, `wien`, `abs_magnitude`, `distance_modulus`, `spectral_class`, `schwarzschild_radius`, `escape_velocity`

Orbital: `orbital_period`, `orbital_velocity`, `hill_sphere`, `roche_limit`, `synodic_period`, `planet(name)`

Solar: `solar_flux`, `solar_wind_pressure`, `sunspot_cycle`, `parker_spiral_angle`, `solar_activity()`

Cosmology: `hubble_distance(z)`, `redshift_velocity(z)`, `lookback_time(z)`, `critical_density()`

Exoplanet: `transit_depth`, `habitable_zone`, `equilibrium_temp`; Pulsar: `pulsar_spindown(P_s, Pdot)`

Gem v0.1.0 2026-04-16

mobl — Mobile & Cross-Platform PWA

```
mobl phone("device_name")
phone.dictate(spoken_text)         # NLP parse -> JSON {title, note, tags}
phone.make_feature(lat, lon, text) # GPS + dictation -> GeoJSON Feature
```

Routes: `/` → PWA HTML, `/log` → POST `{lat,lon,text}`, `/data` → GET GeoJSON FeatureCollection

Other Key Modules

- **fin**: `ticker`, `bs_price`, `greeks`, `high_yield_bonds/etfs/equities`, date/calendar helpers
- **bsm**: inherits `fin` + `price_american(symbol, type, duration)`
- **chart**: `plot(traces, [layout])`, `show(path)`, `server(path, [port])`
  Comparisons: C++=`matplotlib-cpp/ROOT` · Python=`plotly/matplotlib` · Julia=`Plots.jl/Makie.jl` · Rust=`plotters crate`
- **data**: `read_csv(path)`, `mean(v)`, `std(v)`
- **rex**: `match`, `find`, `findall`, `groups`, `sub`, `gsub`, `split`, `count` (ECMAScript, optional flags)
  Comparisons: C++=`std::regex` · Python=`re` · Julia=`Regex` · Go=`regexp` · Ruby=`=~/gsub` · Rust=`regex crate`
- **algo**: `add`, `quicksort`, `sort`, `now()`, `date_add`, `date_diff`
- **tcp**: `listen`, `accept`, `connect`, `send`, `recv`, `close`, `nic()`, `routes()`
  Comparisons: C++=`POSIX socket` · Python=`socket.socket` · Julia=`Sockets.listen` · Go=`net.Listen` · Ruby=`TCPServer` · Rust=`TcpListener::bind`
- **thread**: `wait()` — block until complete; `is_finished()` — non-blocking status check (returned by `sys.async()`)
- **www**: `wget(url,file)`, `app(port,[routes])`, `redirect(target,[port])`, `map2d(geojson,output)`
  Comparisons: C++=`cpp-httplib/Crow` · Python=`Flask/FastAPI` · Julia=`HTTP.jl` · Go=`net/http/Gin` · Rust=`actix-web/axum`
- **cdn**: inherits `www` + `start(port,routes_map)`, `stats()`, `purge([path])`, `config(type,content)`
- **k3s**: Docker (`docker_run/ps/build/stop`) + K3s (`k3s_apply/get/pods/nodes/logs`)
  Comparisons: C++=`Docker SDK` · Python=`docker-py/kubernetes` · Go=`docker/client/k8s.io/client-go` · Rust=`bollard/kube-rs`
- **vm**: `init`, `up`, `ssh`, `status`, `halt`, `destroy`
- **seo**: `index(urls, output_path)` — crawl URLs, extract SEO signals, write JSON; `analyze(index_path)` — print SEO report
- **trek**: `new(path)`, `add(path,lat,lon,[title],[note])`, `edit`, `remove`, `load`, `show(path,[port])` (OSM/Leaflet), `export_gpx`, `stats(path)`
  Comparisons: C++=`GDAL/libgpx` · Python=`gpxpy/folium` · Julia=`GPX.jl` · Go=`tkrajina/gpxgo` · Rust=`gpx crate`
- **mobl**: `phone(name)`, `dictate(text)`, `make_feature(lat,lon,text)`
  Comparisons: C++=`Qt` · Python=`Flask+SpeechRecognition` · Julia=no built-in · Go=`net/http+custom JS` · Rust=`actix-web+custom JS`
- **drvr**: `linux/win11/macos/android(name)`, `build(target)`, `deploy(target)`
- **nlp**: Natural Language Processing
- **bev**: `data(x,y)`, `fit_line()`, `param(idx)` — Bevington data reduction
- **file**: `write(path, content)`, `exists(path)`
- **zip**: `compress(src, archive)`, `decompress(archive, dest)`
- **img**: `resize(src, w, h, dest)`
- **cpp**: `repl()`, `exec(code)` — C++26 JIT via Cling
- **itr**: `range(n)`, `while(cond_fun, body_fun)`
- **go**, **ruby**, **node**, **rust**: polyglot `run(input)` + language-specific helpers

---

Gem v0.1.0 2026-04-16

Mobile Travel Log App

```
./gem travel_log.g          # start server on :8080
# Android/iPhone → http://<host-ip>:8080
# macOS/Linux/Win11 → http://localhost:8080
```

Uses the `mobl` builtin + `travel_log.html` (PWA). Voice dictation → AI NLP → GPS pin → live GeoJSON → Plotly map.

Getting Started

Prerequisites

- **macOS/Linux**: `g++` (supporting C++2b/C++26), `make`, `python3` (3.14+), `libreadline-dev`. Python 3.14 headers required for embedded polyglot runtime.
- **Windows 11**: MSYS2 with `mingw-w64-x86_64-gcc`, `make`, and `mingw-w64-x86_64-python`. Ensure `C:\msys64\mingw64\bin` is in your PATH.

Build and Run

```
make all
./gem tutorial/01_basics.g
```

On Windows, the binaries will be produced as `gem.exe` and `gem_test.exe`.

Or enter the REPL:

```
./gem
```

Developed with Gemini CLI and Kiro.

Gem v0.1.0 · 2026-04-16